

PACKT CERTIFICATION

Graph Engineering for AI Agents

Build, test, and validate reliable AI agent workflows using graphs

25
OCT

09:00 AM ET
Live, Virtual &
Practical

[REGISTER NOW](#) **AGENTIC ENGINEERING**
by packt

Denis Rothman

AI Systems Architect
Author of cutting edge AI books
AI Guru and Visionary

Turn agent workflows into graphs you can
validate, measure and certify before they
run.



Sun, 25 Oct 2026

14:30 GMT, 20:00 IST



Live on Airmet

RECORDING INCLUDED



Certification

PACKT ENDORSED

Why Agent Demos Break in Production

Most agent systems are a **chain of prompts and tool calls**. Nothing can be checked until tokens are already burning.

PROPERTY	PROMPT CHAIN	EXPLICIT GRAPH
Dependencies	Implied in prose	Edges you can inspect
Failures	Found at runtime	Found before the run
Reliability	A feeling	A number
Cost	Known at invoice	Budgeted per node
Audit	Grep the logs	Trace vs the plan

WHAT BREAKS WITHOUT A GRAPH

SYMPTOMS

Step 4 runs before step 2. A tool nobody approved gets called. Half the work missing, run marked done.

GRAPH VIEW

A missing edge. A node with no policy. A terminal node never reached. All testable before execution.

 If you cannot draw it as nodes and edges, you cannot test it. You can only watch it.

An Agent Plan Is a DAG

Nodes are agents or tools. An edge means **must finish before**. No path ever loops back.

LEVEL 0

fetch

LEVEL 1

parse

enrich

LEVEL 2

write

TERMINAL

review

```
plan_to_dag.py
```

```
import networkx as nx

plan = {                                # the plan is data, not prose
    "fetch": {"deps": []},
    "parse": {"deps": ["fetch"]},
    "enrich": {"deps": ["fetch"]},
    "write": {"deps": ["parse", "enrich"]},
    "review": {"deps": ["write"], "terminal": True},
}

g = nx.DiGraph()
for node, spec in plan.items():
    g.add_node(node, **spec)
    for dep in spec["deps"]:
        g.add_edge(dep, node) # edge = "must finish before"
```



Once the plan is a graph object, every later question is a graph query.

Learn This Hands On, in One Day

Everything in this deck comes from a live workshop for engineers taking agents **beyond demos**.



Denis Rothman, AI Systems Architect

Four decades in AI. Author of RAG-Driven Generative AI and six more AI books.

5

HOURS LIVE

5

HANDS-ON LABS

1

CERTIFICATION

**Sun, 25 Oct 2026**

DATE

**14:30 GMT, 20:00 IST**

START TIME

**Live on Airmeeet**

RECORDING INCLUDED

**Intermediate**

PYTHON 3.10+, COLAB

- ✓ Live Q&A with the instructor
- ✓ Packt endorsed certification
- ✓ Notebook and reference solutions
- ✓ Grading harness and scorecard

\$169.99 per ticket, down from \$249.99

Ticket link is in the caption



Validate the Plan Before It Runs

Three structural bugs cause most broken plans. All three are **graph properties**, caught before a single token is spent.

```
validate.py

def validate(g, plan):
    errors = []
    if not nx.is_directed_acyclic_graph(g):
        errors.append(f"cycle: {nx.find_cycle(g)}")
    for n in g.nodes:
        if g.in_degree(n) == 0 and g.out_degree(n) == 0:
            errors.append(f"orphan: {n}") # never reached
    for n, spec in plan.items():
        for dep in spec["deps"]:
            if dep not in plan:
                errors.append(f"broken ref: {n} needs {dep}")
    return errors # empty list means safe to compile
```

CYCLE	A needs B needs A. The run never ends.
ORPHAN	A node with no edges. Nothing ever calls it.
BROKEN REF	A dependency that does not exist. Usually a renamed step.



LLM generated plans produce all three. Validate every plan, not just the hand written ones.

Execution Levels Come for Free

A topological sort groups nodes into levels. Every node in a level has its inputs ready, so the level runs **in parallel**.

```
levels.py

levels = list(nx.topological_generations(g))
# [['fetch'], ['parse', 'enrich'], ['write'], ['review']]

async def run_plan(levels):
    for depth, nodes in enumerate(levels):
        results = await asyncio.gather(
            *(run_agent(n) for n in nodes) # one level
        )
        if any(r.failed for r in results):
            return depth # stop before bad output spreads
    return None
```

DEPTH	Level count is the critical path, your minimum latency.
WIDTH	The widest level is peak concurrency and rate limit pressure.
STOP POINT	The returned depth says exactly which level broke.

 Parallelism is a property of the graph, not a feature you add later.

Measure Reliability as a Number

Give every node a measured success rate. All required nodes must pass, so **the rates multiply**.

EVERY NODE AT 95 PERCENT, INDEPENDENT FAILURES

5 NODES

77%

10 NODES

60%

20 NODES

36%

40 NODES

13%

```
reliability.py

p = {"fetch": 0.99, "parse": 0.95, "enrich": 0.90,
     "write": 0.92, "review": 0.97} # measured per node

def workflow_reliability(g, p):
    r = 1.0
    for n in nx.topological_sort(g):
        r *= p[n] # every required node must pass
    return r

print(workflow_reliability(g, p)) # 0.755
# five nodes that "mostly work" fail one run in four
```

SOURCE

Run each node alone on 50 to 100 eval cases. Count passes.

RE-MEASURE

After any prompt, model or tool change on that node.



The weakest node sets the ceiling. Fix enrich first, not the prompts around it.

Exception Budgets, Not Try Except

Borrowed from SRE error budgets. Decide up front **how much failure a run may absorb**, then check after every node.

```
budget.py

budget = {
    "max_failed_nodes": 1,      # tolerated before abort
    "max_retries_total": 3,
    "max_tokens": 120_000,
}

spent = {"failed": 0, "retries": 0, "tokens": 0}

def within_budget(spent, budget):
    return (spent["failed"] ≤ budget["max_failed_nodes"]
            and spent["retries"] ≤ budget["max_retries_total"]
            and spent["tokens"] ≤ budget["max_tokens"])

# evaluated after every node, so the abort is early and cheap
```

WITHOUT A BUDGET

Each node retries alone. A flaky tool retries five times and the invoice arrives.

WITH A BUDGET

Retries are a shared, finite resource. The run aborts at node 3 with a reason.



Tokens belong in the budget. Success at 4x the planned cost is not success.

Reliability vs Token Cost

Every repair that raises reliability raises cost. The graph lets you **price each repair per node** before choosing.

REPAIR	RELIABILITY GAIN	TOKEN COST	LATENCY
Retry once	Large	Up to 2x	Up to 2x
Retry twice	Small	Up to 3x	Up to 3x
Verifier node	Large	+1 call	+1 hop
Stronger model	Varies	3x to 10x	Higher
Split the node	Large	About 1x	Same

```
repairs.py

def with_retries(p, k):
    # k attempts, each succeeding with probability p
    return 1 - (1 - p) ** k

print(with_retries(0.90, 1)) # 0.900 1x tokens
print(with_retries(0.90, 2)) # 0.990 up to 2x
print(with_retries(0.90, 3)) # 0.999 up to 3x
# retry 1 buys 9 points, retry 2 buys 0.9, same price each
```



Retries only help with independent failures. A malformed input fails the same way three times.

Verification Gates on the Edges

A gate is a deterministic check on an edge. Output that fails it **never** becomes the next node's input.

PRODUCER

write

GATE

schema + rule check

CONSUMER

review

gates.py

```
from pydantic import BaseModel, ValidationError

class Draft(BaseModel):
    title: str
    sources: list[str]

def gate(output: str, min_sources: int = 2) -> bool:
    try:
        d = Draft.model_validate_json(output)
    except ValidationError:
        return False # wrong shape, do not pass
    return len(d.sources) >= min_sources # content rule

g.add_edge("write", "review", gate=gate) # lives on the edge
```



Gates are code, not model calls. Put one on every edge that crosses a trust boundary.

Governance Lives on the Node

Telling the model which tools to use is a request. Tools and token caps **attached to the node** are rules the runtime enforces.

```
policy.py

policy = {
    "fetch": {"tools": {"http_get"}, "max_tokens": 4_000},
    "write": {"tools": set(), "max_tokens": 30_000},
    "review": {"tools": {"read_file"}, "max_tokens": 8_000},
}

def allowed(node, tool, tokens_so_far):
    rule = policy[node]
    return (tool in rule["tools"]
            and tokens_so_far < rule["max_tokens"])

# the node decides what the agent may call, not the prompt
```

TOOL SCOPE	The write node has no tools. Injected text cannot make it call anything.
TOKEN CAP	Per node. One runaway node cannot eat the budget of five.
EDGE POLICY	Which node may hand data to which.



If the system prompt is your only guardrail, your guardrail is a suggestion.

GraphRAG: Vector Seed, Graph Expand

Vector search finds what sounds like the question. The graph finds what **connects to it**. Use both, keep provenance.



```
graphrag.py

seeds = vector_index.search(query, k=5) # semantic entry

hits, frontier = set(seeds), set(seeds)
for hop in range(2): # multi hop
    nxt = set()
    for node in frontier:
        nxt |= set(kg.neighbors(node))
    hits |= nxt
    frontier = nxt - hits # only new nodes

context = [kg.nodes[h]["text"] for h in rank(hits, query)]
provenance = {h: kg.nodes[h]["source"] for h in hits}
# every chunk carries the node id and the source it came from
```



No hop limit and no rank step returns the whole graph. Two hops is the usual ceiling.

Trace the Run, Detect the Drift

A trace records what ran. Drift is the gap between the **planned graph** and the **observed graph**.

```
trace.py

trace = [] # one row per node execution
async def run_node(n, parent):
    t0 = time.perf_counter()
    out = await agent(n)
    trace.append({"node": n, "parent": parent, "ok": out.ok,
                 "tokens": out.tokens,
                 "ms": round((time.perf_counter()-t0)*1000)})
    return out

planned = set(g.edges)
observed = {(r["parent"], r["node"]) for r in trace[1:]}
drift = observed - planned # edges the plan never approved
```

EDGE DRIFT	An observed edge the plan never had.
SKIP DRIFT	A planned edge never observed. Success with a node missing.
COST DRIFT	Tokens per node creeping past the cap.

 Store traces as graphs. Diffing two runs becomes a set operation, not a log search.

Should This Workflow Run at All?

Everything folds into one pass or fail scorecard. Fail **any** row and the workflow is not certified.

```
scorecard.py

scorecard = {
    "structure": validate(g, plan) == [],
    "reliability": workflow_reliability(g, p) >= 0.90,
    "budget": within_budget(spent, budget),
    "gates": all(r["ok"] for r in trace),
    "drift": len(drift) == 0,
}

certified = all(scorecard.values())
# one False and it does not ship, whatever the demo showed
```

FAILURE CATALOG

SYMPTOM	GRAPH DIAGNOSIS
Works in demo, fails at scale	Reliability compounds
Unapproved tool call	No policy on the node
Right answer, wrong source	No provenance
Partial run marked success	No gate, node skipped
Plan and run disagree	Drift nobody measured



The scorecard is the deliverable. "It ran fine yesterday" is not a certification.

Build It With Denis Rothman

Five hours, five labs, one scorecard. Led by **Denis Rothman**, four decades in AI and seven books.

WHO IT IS FOR

ROLES	AI or ML engineer, architect, tech lead.
YOU HAVE	Python 3.10+, Colab, one agent workflow behind you.
YOU WANT	To ship agents on a number, not a feeling.

FIVE LABS, IN THIS ORDER

- ✓ Turn an agent plan into a DAG

✓ Enforce budgets and gates

✓ Trace, detect drift, certify
- ✓ Measure reliability, compare repairs

✓ GraphRAG with provenance

✓ Keep the notebook and harness

 **Sunday, 25 October 2026**
14:30 TO 18:45 GMT

 **\$169.99**
~~\$249.99~~

RECORDING INCLUDED

Stop watching agents run. Start measuring them.
Ticket link is in the caption. Save this post for the code.

